

UNDERACTUATED ROBOTICS

Algorithms for Walking, Running, Swimming, Flying, and Manipulation

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-11-11.

[How to cite these notes, use annotations, and give feedback.](#)

Note: These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

CHAPTER 14

Feedback Motion Planning

1

Throughout these notes we have been developing core ideas from motion planning (especially trajectory optimization and sampling-based planning) and for feedback control (e.g. linear and nonlinear policies). Of course these two lines of work are deeply connected. If you have a trajectory planner that can run fast enough to be evaluated at every control time step (e.g. model-predictive control), then the planner can become a policy. Of course, the opposite is true, too. If you have a policy then you can simulate it forward to generate a trajectory plan.

In the last chapter we started explicitly considering robustness and uncertainty. This complicates the story (but in a beautiful way!). Feedback controllers can be robust, or not, but the planners we've built so far start to struggle. What does it mean for a trajectory to be robust? If we're using $\mathbf{x}(\cdot)$, $\mathbf{u}(\cdot)$ to represent a trajectory plan, then the presence of noise can make the plan infeasible. Planning without considering the noise/uncertainty and then stabilizing that plan can work in the simple cases, but richer connections are possible.

14.1 PARAMETERIZED FEEDBACK POLICIES AS "SKILLS"

When I first introduced the notion of a controller, the job for the controller was to e.g. stabilize a fixed point, or minimize the long-term cost, for all possible initial conditions. Motivated initially by the ability to use linearization and linear optimal control to find feedback controllers even for nonlinear systems, we started to admit that some controllers have a more limited applicability, and to develop tools for estimating their regions of attraction and/or invariant funnels around trajectories.

One of the very original approaches to planning in artificial intelligence research was the [Stanford Research Institute Problem Solver](#) (STRIPS) framework. The original STRIPS framework was for discrete states (actually boolean propositions), with the initial state specified by a set of propositions and the goal state specified by another set of propositions. Most importantly for our discussion here, the "dynamics" were given by a set of actions (or "skills") which were defined by a set of *preconditions* (what propositions must be satisfied for the skill to be executed), and a set of *postconditions* (what propositions will be true if the action is executed).

I'll credit [1] as the paper that first gave me the mental imagery for how regions of attraction and Lyapunov functions can help us lift the notions of continuous control up into this space of "skills" that can potentially be combined with a high-level planning framework like STRIPS. (Ironically, when I asked Dan Koditschek about the sequential composition of funnels idea, he credited Matt Mason; when I asked Matt Mason, he credited Marc Raibert; when I asked Marc Raibert, he said something along the lines of "oh, that's interesting..."). The picture in that paper is simple but powerful: a local controller combined with an estimate of its region of attraction defined the preconditions (also commonly referred to as the *initiation set*) of the skill. An associated Lyapunov function gives us the framework to talk about the postconditions.

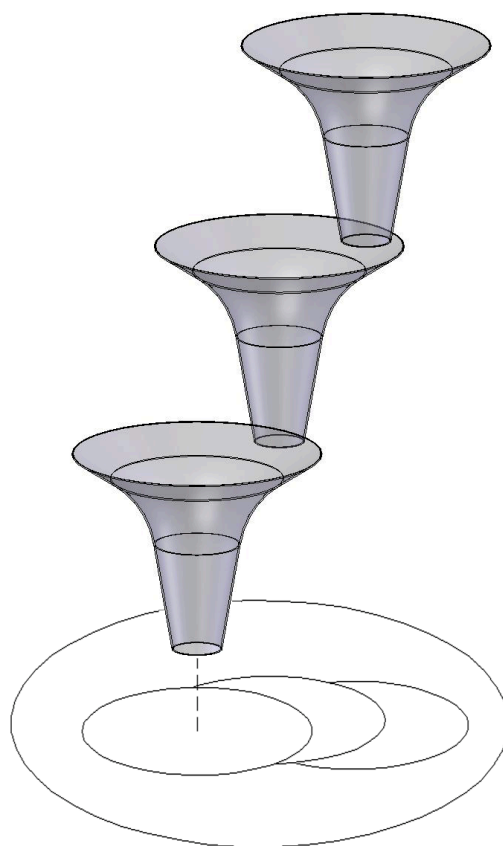


Figure 14.1 - A controller equipped with a Lyapunov function can be thought of as a "skill" with preconditions and postconditions, providing rules for composition.

The field of automated reasoning / task-level planning has evolved dramatically since the introduction of STRIPS. The planning domain definition language (PDDL) [2, 3] extended the STRIPS representation to include more expressive features such as typed variables, conditional effects, and (even temporal logic) quantifiers, and served as the specification language for a long-running and highly influential series of [ICAPS planning competitions](#).

This idea of planning with primitives which are feedback controllers instead of trajectories is potentially very powerful. Let me emphasize one of the key points: *trajectories can't be robust, but funnels can*.

14.1.1 The rules of composition

Imagine I have two controllers, $\pi_1(\mathbf{x})$ and $\pi_2(\mathbf{x})$, each equipped with a corresponding function $V_1(\mathbf{x})$ and $V_2(\mathbf{x})$ which has been certified to satisfy the Lyapunov conditions over their respective sublevel sets, $V_1(\mathbf{x}) \leq \rho_1$, $V_2(\mathbf{x}) \leq \rho_2$.

Let's remember a slightly subtle point. If $\dot{V} \prec 0$, then I'm guaranteed to move down the Lyapunov function, but if $\dot{V} \preceq 0$, then my Lyapunov function only guarantees invariance. For robust stability (again unknown disturbances or modeling errors), it might be unlikely that we can achieve true asymptotic stability. However the cartoon illustration with the funnels above may still be very reasonable. It requires that we have $0 \leq \bar{\rho}_i \leq \rho_i$ such that

$$\bar{\rho}_i \leq V_i(\mathbf{x}) \leq \rho_i \rightarrow \dot{V}_i(\mathbf{x}) \prec 0.$$

In words, we would like to have some (asymptotic) convergence from the initiation set $V_i(\mathbf{x}) \leq \rho_i$ to a smaller invariant set $V_i(\mathbf{x}) \leq \bar{\rho}_i$. Then we can guarantee that a higher-level planner/controller can transition from executing skill i to executing skill j if $V_i(\mathbf{x}) \leq \bar{\rho}_i \rightarrow V_j(\mathbf{x}) \leq \rho_j$. For polynomial Lyapunov functions, one can certify these containment conditions using sum-of-squares optimization with the S-procedure; for quadratic Lyapunov functions this reduces nicely to ellipsoidal containment.

The other form of Lyapunov functions which we have studied are the [time-varying Lyapunov functions](#) associated e.g. with stabilizing a trajectory. In this case, if the Lyapunov conditions, $\dot{V}(t, \mathbf{x}) \leq \dot{\rho}_i(t)$, are satisfied for all $\{t, \mathbf{x} | V_i(t, \mathbf{x}) = \rho_i(t), t_0 \leq t \leq t_f\}$, and the underlying dynamics are time-invariant, then it is sufficient to satisfy the containment conditions $V_i(t_i, \mathbf{x}) \leq \rho_i(t_i) \rightarrow V_j(t_j, \mathbf{x}) \leq \rho_j(t_j)$ for any valid t_i, t_j .

Example 14.1 (Pendulum: transitioning from swing-up to balance)

14.1.2 Parameterized controllers and Lyapunov functions

We can similarly extend our notion of a continuous feedback skills to include parameterized skills/funnels, for instance when the skills are "goal-conditioned".

Example 14.2 (Koditschek's juggling robots)

Polynomial parameterizations and SOS. But there are a few special cases that can work out particularly nicely...

Equivariance (cyclic coordinates) [4].

State-dependent Riccati equations (SDRE) [5].

Parameterizing via a nominal trajectory.

14.2 PROBABILISTIC FEEDBACK COVERAGE

There is a nice way to combine the ideas from sampling-based motion planning with this idea of (guaranteed) sequential composition with funnels [6].

14.3 ONLINE PLANNING

REFERENCES

1. R. R. Burridge and A. A. Rizzi and D. E. Koditschek, "Sequential Composition of Dynamically Dexterous Robot Behaviors", *International Journal of Robotics Research*, vol. 18, no. 6, pp. 534-555, June, 1999.
2. Constructions Aeronautiques and Adele Howe and Craig Knoblock and ISI Drew McDermott and Ashwin Ram and Manuela Veloso and Daniel Weld and David Wilkins SRI and Anthony Barrett and Dave Christianson and others, "PDDL| the planning domain definition language", *Technical Report, Tech. Rep.*, 1998.
3. Patrik Haslum and Nir Lipovetzky and Daniele Magazzeni and Christian Muise and Ronald Brachman and Francesca Rossi and Peter Stone, "An introduction to the planning domain definition language", Springer , vol. 13, 2019.
4. Anirudha Majumdar, "Funnel Libraries for Real-Time Robust Feedback Motion Planning", PhD thesis, Massachusetts Institute of Technology, Jun, 2016. [[link](#)]
5. Anirudha Majumdar and Mark Tobenkin and Russ Tedrake, "Algebraic Verification for Parameterized Motion Planning Libraries", *Proceedings of the 2012 American Control Conference (ACC)* , pp. 8, June, 2012. [[link](#)]
6. Russ Tedrake and Ian R. Manchester and Mark M. Tobenkin and John W. Roberts, "LQR-Trees: Feedback Motion Planning via Sums of Squares Verification", *International Journal of Robotics Research*, vol. 29, pp. 1038--1052, July, 2010. [[link](#)]

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

